TransferIa

Manual técnico

Arquitectura, módulos, flujo de procesamiento, formatos y pruebas

Versión del 26 de octubre de 2026

Contenido

1. Visión general 3
2. Estructura del proyecto 3
3. Modelos de IA 4
4. Pipeline de un frame 4
4.1 Detección 4
4.2 Pegado (pegar_cara) 4
4.3 GPEN (Mejorador_Cara) 5
4.4 Oclusión y nitidez (xseg.py) 5
4.5 Partes del destino (conservar_partes) 5
4.6 Corrección de la detección de caras inclinadas (26 de octubre de 2026) 5
4.7 Caras en primer plano (26 de octubre de 2026) 8
5. Asignación de caras 9
5.1 Por posición 9
5.2 Por identidad 9
6. Video (core/Proceso_Video.py) 10
6.1 Videos de ritmo variable o con el audio corrido 10
6.2 Video de prueba 10
7. Carpetas, modelos, configuraciones y ZIP 10
7.1 Carpeta como destino (Proceso_Carpeta) 10
7.2 Modelos de cara (.npy) 10
7.3 Configuraciones (JSON) 11
7.4 ZIP con contraseña (core/Empaquetar.py) 11
8. Interfaz y CLI 11
8.1 app.py 11
8.2 cli.py 11
9. Rendimiento 12
10. Pruebas realizadas 12
11. Dependencias y entorno 12
12. Limitaciones conocidas y mejoras pendientes 13
13. Cómo extender 13

1. Visión general

TransferIa es una aplicación en **Python** para cambio de cara en imágenes y videos. La lógica está en el paquete `core/`, que usan por igual la **interfaz** (`app.py`, Tkinter) y la **CLI** (`cli.py`, argparse). El cuaderno `TransferIa_Colab.ipynb` es una capa de formularios sobre la CLI.

Los modelos de IA se ejecutan con **ONNX Runtime** (CUDA si hay GPU NVIDIA; si no, CPU). El video se lee con OpenCV y se escribe con **FFmpeg** a través de una tubería, sin imágenes intermedias en disco.

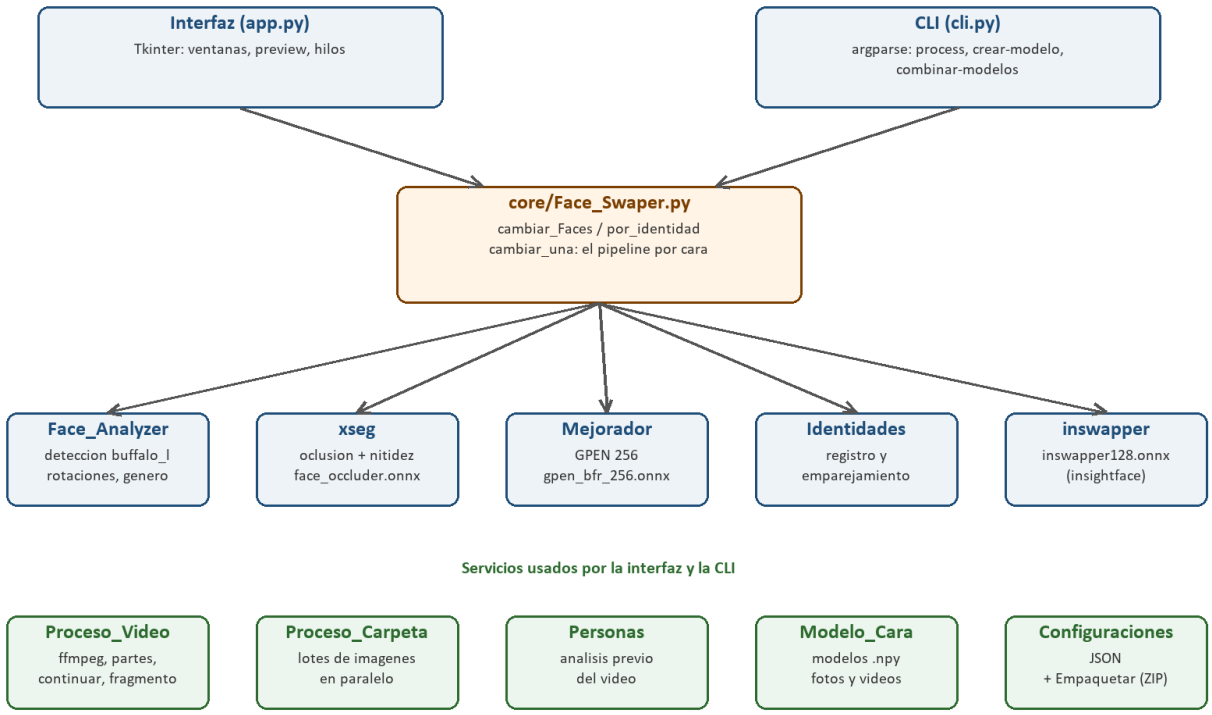


Figura 1. Arquitectura: interfaz y CLI sobre el núcleo (core) y sus servicios.

2. Estructura del proyecto

Ruta	Contenido
app.py	Interfaz gráfica: FaceSwapApp, DropFrame, PreviewWindow, VentanaFragmento (~2.400 líneas)
cli.py	Línea de comandos: process, crear-modelo, combinar-modelos (~870 líneas)
core/Face_Swaper.py	Swapper_Faces: el pipeline de cambio de cara por cara y por frame
core/Face_Analyzer.py	Analyzer_Faces: detección (buffalo_l), caras rotadas, filtro de género, orden
core/xseg.py	DFL_XSeg_Model: máscara de oclusión, modos de oclusión y filtros de nitidez
core/Mejorador.py	Mejorador_Cara: GPEN 256 y el tamaño mínimo de cara
core/Identidades.py	Registro_Identidades: calidad de cara, referencias y emparejamiento
core/Personas.py	Análisis previo del video para el modo identidad (y su caché)
core/Modelo_Cara.py	Modelos de cara .npy: crear (fotos y videos), combinar, cargar, listar, eliminar
core/Proceso_Video.py	Escritor_Video (FFmpeg), Trabajo_Video (.partes, continuar), procesar_video, unir
core/Proceso_Carpeta.py	Carpeta de imágenes como destino, en paralelo
core/Configuraciones.py	Configuraciones guardadas: validación, guardar, cargar, listar
core/Empaquetar.py	ZIP con contraseña AES-256 (pyzipper) con verificación
TransferIa_Colab.ipynb	Cuaderno de Colab con formularios
models/	Modelos ONNX (no versionados)
configuraciones/, output/, src/	Datos del usuario (src/ y output/ fuera de git)

Archivos sin uso: `cambiador.py`, `face_enhancer.py`, `gpen_model.py`, `config.py` (vacío), `core/Face_Enhancer.py` y `core/xseg copy.py` son de versiones anteriores: ningún módulo actual los importa.

3. Modelos de IA

Modelo	Tamaño	Función	Lo usa
buffalo_l (insightface)	~326 MB, 5 modelos	Detección, 106 puntos, 68 puntos 3D, género/edad y reconocimiento (embedding de 512)	Face_Analyzer
inswapper128.onnx	554 MB	Cambio de cara en un recorte alineado de 128 px	Face_Swaper
face_occluder.onnx	70 MB	Segmentación de la cara visible (XSeg), entrada 256×256	xseg
gpen_bfr_256.onnx	76 MB	Restauración de caras (GPEN) a 256 px	Mejorador

Las sesiones se crean con los proveedores `CUDAExecutionProvider` y `CPUExecutionProvider`, en ese orden: con GPU NVIDIA y `onnxruntime-gpu` se usa CUDA; si no, CPU. Las sesiones son atributos de clase (se cargan una sola vez) y las inferencias de XSeg van protegidas con un lock.

4. Pipeline de un frame



Figura 2. Pasos que se aplican a cada cara de cada frame (Swapper_Faces.cambiar_una).

4.1 Detección

- `Analyzer_Faces.obtener_caras`: detección a 480 px (640 al crear modelos), filtro de género y orden de izquierda a derecha.
- Si no encuentra caras o alguna está inclinada más de 45°, vuelve a buscar con la imagen girada 90°, 180° y 270°. Si la misma cara aparece en varios giros se elige la que quedó **más derecha en su giro** (antes, la de mayor `det_score`: a 47° elegía el giro de 90°, con los puntos cuatro veces más lejos).
- Las caras inclinadas se enderezan y se detectan otra vez (refinado). Entre `REFINAR_DESDE` (10°) y `REFINAR_COMPLETO` (20°) se interpolan los puntos, el recuadro y los 106 puntos entre la detección normal y la refinada. Antes el refinado empezaba de golpe a los 30° y la cara saltaba entre frames cerca de ese ángulo.
- `Analyzer_Faces.CARAS_INCLINADAS = False` (casilla «Caras inclinadas», `--sin-inclinadas`) usa la detección original de insightface.

La corrección de estos dos puntos y su verificación están en la sección 4.6.

4.2 Pegado (pegar_cara)

- inswapper genera la cara en el recorte alineado de 128 px.

- Máscara `normal`: el pegado de insightface. `amplia` y `maxima`: máscara propia en el recorte (`MASCARAS`: recorte de borde y difuminado: normal 0.10/0.05, amplia 0.05/0.035, máxima 0.02/0.025).
- Corrección de color (suave/completa) y conservación de maquillaje (`SIGMA_MAQUILLAJE` 0.03) en el recorte.
- La cara se lleva al frame con la transformación inversa y se mezcla con la máscara.

4.3 GPEN (Mejorador_Cara)

Restaura la cara a 256 px y la mezcla con `MEZCLA` (0.8 por defecto, o la fuerza elegida). Las caras más chicas que el mínimo (`96` px por defecto, o un porcentaje del lado menor de la imagen) no se mejoran. Va antes de la oclusión, así pelo y manos no pasan por el mejorador.

4.4 Oclusión y nitidez (xseg.py)

- La cara se alinea con sus 5 puntos (`USAR_ALINEACION`), XSeg genera la máscara de cara visible y vuelve al ángulo original.
- La máscara se difumina en proporción a la distancia entre ojos (sigma 0.09·d y 0.19·d, mezcla 80/20).
- **Modo de oclusión** (`MODOS_OCLUSION`): `debil` usa la máscara difuminada tal cual; `adaptable` toma el mínimo entre la máscara difuminada y la del modelo (el difuminado no puede pintar cara sobre el objeto); `fuerte` además la contrae: `(m.clip(0.5, 1) - 0.5)·2`.
- **Nitidez**: máscara de enfoque solo sobre la luz (canal L de LAB), con radio según el ancho de la cara (ancho/128). `natural` y `strong` usan fuerza fija (`NITIDEZ_FIJA` 0.8 y 1.6). `adaptativo` calcula la fuerza para llegar al 90 % del detalle del destino (`NITIDEZ_OBJETIVO`, tope `NITIDEZ_MAXIMA` 3) y agrega grano hasta el 85 % (`GRANO_OBJETIVO`).
- Sin oclusión, el filtro se aplica dentro de la zona de pegado (`Swapper_Faces.aplicar_nitidez`).

4.5 Partes del destino (conservar_partes)

Con los 106 puntos se arma una máscara convexa de ojos (agrandada ×1.35), boca (×1.2) y nariz (×1.15); sin ellos, elipses sobre los 5 puntos. Se difumina con un núcleo proporcional a la distancia entre ojos y se mezcla con el frame original. La **frente** usa `mascara_frente` (desde el borde de las cejas hacia arriba, en el sistema de ejes de la cara) e `igualar_tono_frente` (diferencia de mediana en LAB medida en el entrecejo).

Optimización: todo se calcula solo en el recuadro que contiene las formas (con margen de 2 radios del difuminado), no en el frame entero. Si el recuadro toca el borde izquierdo o derecho se usan filas completas, porque OpenCV difumina el borde por otro camino según el ancho. El resultado es idéntico píxel a píxel (verificado en 4.200 casos).

Partes	1080p antes	1080p ahora	4K antes	4K ahora
Ojos	236 ms	4 ms	781 ms	11 ms
Ojos + boca + nariz	192 ms	5 ms	621 ms	19 ms
Frente	178 ms	15 ms	655 ms	58 ms
Todo	177 ms	21 ms	667 ms	77 ms

4.6 Corrección de la detección de caras inclinadas (26 de octubre de 2026)

Síntoma

En videos donde la cabeza se inclina, la cara pegada se veía «inconsistente» en algunos ángulos: saltaba de un frame a otro aunque la cabeza se moviera suave.

Cómo se encontró

Se armó una prueba medible: una cara generada por IA girada grado a grado. Como se conoce el giro exacto de cada frame, se sabe dónde deberían estar los 5 puntos (ojos, nariz, comisuras) y se puede medir el **error** (distancia media a su posición exacta, en % de la distancia entre ojos) y el **salto** (cuánto cambia el error de un grado al siguiente, que en video se ve como temblor). Se compararon tres versiones: la detección original de insightface, la de antes del arreglo (último commit) y la corregida.

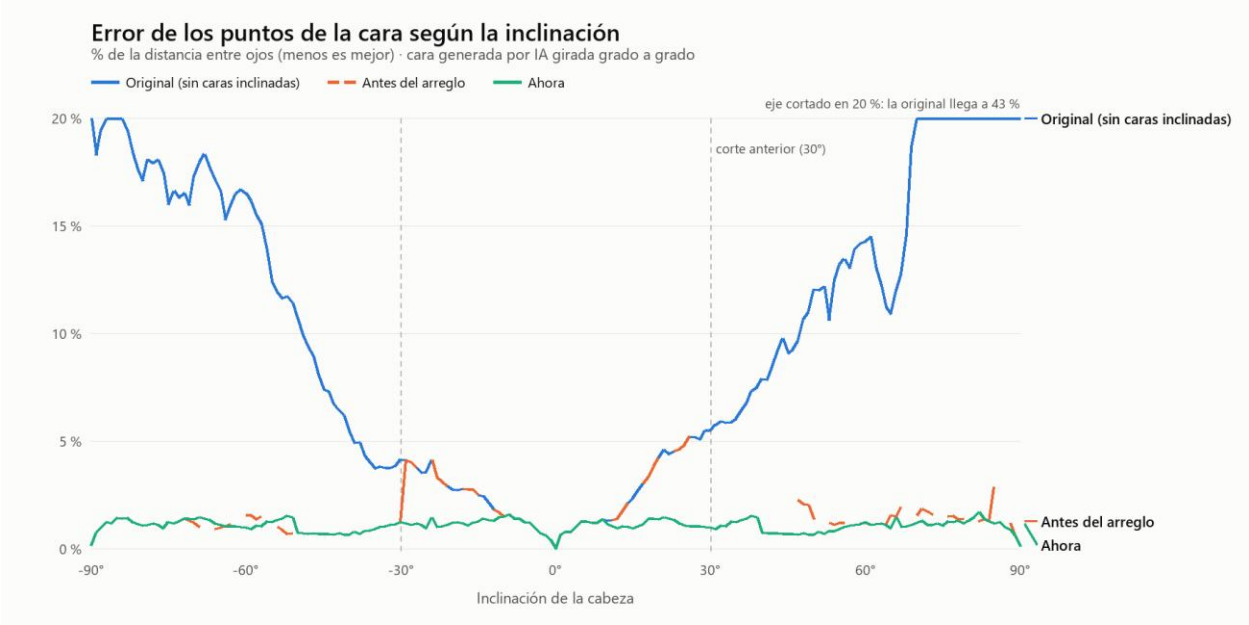


Figura 3. Error de los puntos según la inclinación. La versión anterior (naranja, punteada) salta a los ±27-30° y otra vez a los 47°; la corregida (verde) es estable en todo el rango.

Causa 1: el corte de 30°

El refinado (enderezar la imagen y volver a detectar) se aplicaba solo a las caras inclinadas más de 30°. Hasta 26° se usaban los puntos sin refinar (error ~5 %) y desde 27°, que la inclinación medida ya pasa de 30°, los refinados (~1 %). Con la cabeza oscilando cerca de ese ángulo, cada frame usaba uno u otro y la cara pegada saltaba un 4 % de la distancia entre ojos.

```
# Antes
if abs(angulo) <= UMBRAL_REFINAR_INCLINACION:   # 30
    return cara                                  # sin refinar
...
return mejor                                     # refinada, de golpe

# Ahora: entre 10 y 20 grados se mezclan de a poco
if abs(angulo) <= REFINAR_DESDE:                 # 10
    return cara
peso = clip((abs(angulo) - REFINAR_DESDE) / (REFINAR_COMPLETO - REFINAR_DESDE), 0, 1)
mejor.kps = (1 - peso) * cara.kps + peso * mejor.kps      # idem bbox y 106 puntos
```

Con el refinado empezando a 10°, donde los puntos normales y los refinados casi coinciden, la transición no se nota, y como además es gradual no hay ningún salto.

Causa 2: la elección entre giros a los 47°

Si una cara está inclinada más de 45°, se busca también en la imagen girada 90°, 180° y 270°, y la misma cara aparece varias veces. Se elegía la de mayor `det_score`. A los 47° la detección del giro de 90° tenía un puntaje apenas mayor (0,770 contra 0,764 del giro de 270°), pero sus puntos estaban mucho peor (24 % de error contra 6,7 %), y el refinado partía de una mala inclinación.

```
# Antes: mayor puntaje
for c in sorted(caras, key=lambda c: c.det_score, reverse=True): ...
```

```
# Ahora: la que quedó más derecha en la imagen donde se detectó (ahí el detector es más preciso);
# a igual inclinación (de a 5 grados), el mayor puntaje
c.inclinacion_deteccion = abs(self.inclinacion(c))      # medida antes de volver al frame original
orden = lambda c: (round(c.inclinacion_deteccion / 5.0), -c.det_score)
```

Verificación



Figura 4. Puntos detectados (naranja) contra su posición exacta (círculo verde). La original se aleja al inclinarse (29 % a 120°, 56 % boca abajo); la anterior fallaba a los 26° y a los 47°; la corregida queda en 0,1-1,1 % en todos los ángulos.

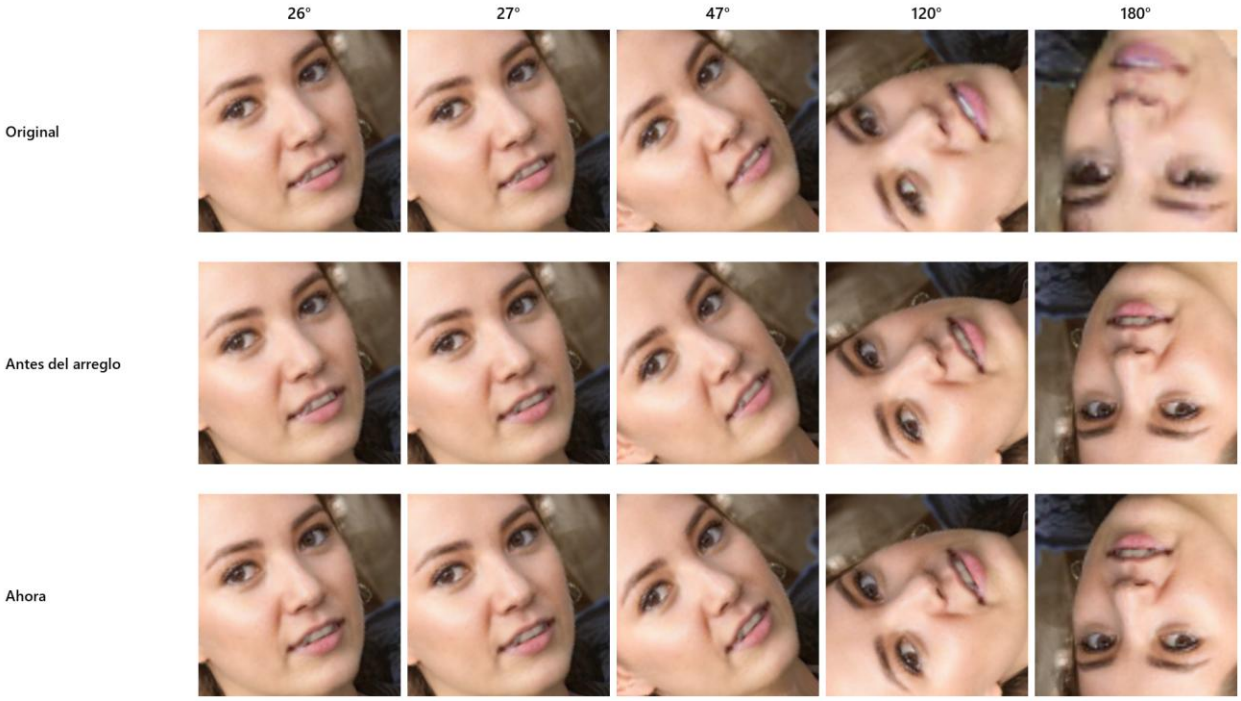


Figura 5. Cambio de cara resultante (fuente: otra cara generada por IA). Con la detección original, a 120° y 180° la cara queda desalineada (ojos y boca dobles); con la corregida sigue el giro.

Medida (cara girada grado a grado)	Original	Antes del arreglo	Ahora
Error medio −90° a +90° (% de la distancia entre ojos)	11,3 %	1,5 %	1,1 %
Error máximo −90° a +90°	43,2 %	5,2 %	1,7 %
Mayor salto entre grados consecutivos	9,4 %	4,2 %	0,7 %
Error medio −180° a +180° (boca abajo incluido)	24 %	—	1,0 %

También se generó un video de una cabeza girando 360° y se procesó con y sin la detección de caras inclinadas: con ella se cambió la cara en los 240 frames, sin ella en 238 (pierde la cara cerca de los 180°) y desalineada al inclinarse.

Compatibilidad

- Nueva opción para volver a la detección original: casilla «Caras inclinadas» de la interfaz, `--sin-inclinadas` en la CLI, `caras_inclinadas` en las configuraciones (`Analyzer_Faces.CARAS_INCLINADAS`).
- La firma de los videos a medias lleva `deteccion: 2`: un video empezado antes del arreglo no se continúa (la cara se ubicaría distinto a mitad del video); hay que empezarlo de cero.

4.7 Caras en primer plano (26 de octubre de 2026)

Síntoma

Una cara que llena casi todo el cuadro no se detectaba: en un video, esos frames quedaban sin cambiar, y no se podía crear un modelo con un primer plano. Al arreglarlo apareció un segundo defecto: un borde oscuro en el contorno de la imagen.

Causa 1: el tamaño máximo del detector

El detector (SCRFD de buffalo_l) solo reconoce caras hasta cierto tamaño respecto de su entrada. Con caras generadas por IA recortadas cada vez más cerca:

La cara ocupa del ancho	Procesar (480 px), antes	Crear modelo (640 px), antes	Ahora (ambos)
40 %	5/5	5/5	5/5
55 %	5/5	2/5	5/5
70 %	3/5	0/5	5/5
80 %	0/5	0/5	5/5
100-140 %	0/5	0/5	5/5 (error 2,6-7 %)

Solución: `Analyzer_Faces._detectar_primer_plano`: si la detección normal no encuentra ninguna cara, prueba con la imagen dentro de un marco (`BORDES_PRIMER_PLANO`): margen de medio lado mayor con borde repetido (la imagen queda a la mitad: lo más preciso hasta el 100 %), luego un lado entero (a un tercio, hasta 125 %) y por último borde negro (140 %). Las coordenadas se devuelven al frame original. Si la detección normal ya encontró caras no se hace nada más, así que el tiempo y el resultado de los casos normales no cambian.

Causa 2: el relleno negro del recorte

inswapper recorta la cara alineada en un cuadrado de 128 px (`face_align.norm_crop2`). Si ese cuadrado se sale del frame, lo de afuera se rellena con negro, el modelo lo reproduce y al pegar quedaba un borde oscuro: medido, el borde del resultado era un 25-50 % más oscuro que el original.

Solución: `Swapper_Faces._margen_recorte` calcula cuánto se sale el recorte; si se sale, `pegar_cara` trabaja sobre el frame con ese margen de borde repetido (`_cara_desplazada` mueve los puntos) y recorta el resultado. Después del arreglo el brillo del borde quedó igual al original (±2).



Figura 6. Cara que llena el cuadro: antes no se detectaba; con solo la primera corrección aparecía el borde oscuro; con las dos, el resultado es limpio.

Verificación

- 5 caras generadas por IA a 9 acercamientos (40 % a 140 % del ancho): todas detectadas, al procesar y al crear modelos, con y sin «Caras inclinadas».
- Crear un modelo con un primer plano que antes fallaba: ahora se crea.
- Con la cara dentro del frame, el resultado es idéntico píxel a píxel al de la versión anterior.

5. Asignación de caras

5.1 Por posición

Los índices se emparejan de a pares (`zip(indices_src, indices_dst)`). Un par cuyo índice no existe en ese frame se salta, así una misma fuente puede ir a varias caras (`0,0 → 0,1`) y un frame con menos personas no falla.

Limitación: La CLI no valida antes de empezar que los índices fuente existan en la foto fuente (la interfaz sí lo hace). Un índice inexistente hace que ese par se salte en todos los frames sin aviso.

5.2 Por identidad

`Registro_Identidades` guarda una o varias referencias (embeddings normalizados) por identidad y `asignar` empareja de forma voraz por mayor similitud coseno (umbral 0.35), sin repetir caras ni identidades. Con referencias múltiples, cuenta la toma más parecida.

Análisis previo de personas (core/Personas.py)

- Recorre el video a `FPS` = 1 frame por segundo, como máximo `MAX_FRAMES` = 300 repartidos en todo el video (`Modelo_Cara.frames_video`). Configurable: `--personas-fps`, `--personas-max`.
- Agrupa las caras por persona (similitud con el promedio del grupo ≥ 0.35, una persona por cara en cada frame). Las caras borrosas o de perfil (calidad 0) no crean grupos.
- Une grupos con promedios parecidos (≥ 0.45) que nunca aparecen en el mismo frame.
- Ordena por apariciones (0 = la que más aparece) y guarda por persona el promedio y hasta 5 tomas de buena calidad con similitud entre sí < 0.8 (variadas).
- Caché en `<salida>/<video>_personas/personas.json` + `persona_N.jpg`, con clave de video (ruta, tamaño, fecha), género, fps, máximo y versión.
- Durante el proceso las identidades quedan fijas (`Registro_Identidades.fijar`): todas las personas tienen referencia, así los extras no se confunden con las personas pedidas.

6. Video (core/Proceso_Video.py)

- Un hilo lector lee los frames con OpenCV y llama a `preparar(frame, numero)` en orden (ahí se hace el registro de identidades); los cambios corren en un pool de `hilos` hilos.
- `Escritor_Video` manda los frames a FFmpeg por una tubería (`rawvideo bgr24`, `-r fps`), en H.264 con GOP de 2 segundos, en partes `.mkv` o `.mp4` fragmentado que aguantan cortes.
- `Trabajo_Video` guarda el estado en `<salida>.partes/`: partes terminadas, frames hechos, estado de identidades y una **firma** con las opciones que cambian la imagen. Para continuar, la firma y el video deben coincidir.
- `unir` concatena las partes sin recomprimir y les pone el audio original (copiado, o AAC si no entra en el formato). Con fragmento, recorta el tramo de audio correspondiente.

6.1 Videos de ritmo variable o con el audio corrido

Los frames se escriben a ritmo fijo y el audio se pega desde el principio, así que el resultado solo coincide con el original si este tiene ritmo fijo y audio e imagen empiezan juntos. `problemas_de_tiempo` lo revisa con ffprobe (`r_frame_rate` contra `avg_frame_rate`, diferencia > 1 %; `start_time` de video y audio, diferencia > 20 ms). Si hay un problema, `normalizar` crea `<salida>.partes/normalizado.mkv`: el tramo pedido cortado por tiempo, a ritmo fijo (`fps=…:start_time=0`, repite el primer frame si a la imagen le falta el principio) y con el audio alineado (`aresample=async=1:first_pts=0`, PCM). Se procesa esa copia y el audio final sale de ella. Al continuar un video cortado se reutiliza.

Video de origen	Desfase máximo antes	Desfase máximo ahora
Ritmo fijo	1 ms	1 ms (no se normaliza)
Ritmo variable, completo	428 ms	21 ms
Ritmo variable, fragmentos	415-468 ms (uno fallaba)	19-21 ms
Audio e imagen corridos 0,5 s	cambiaba el desfase respecto del original	igual que el original

Medido con videos sintéticos con un destello y un pitido en cada segundo.

6.2 Video de prueba

`procesar_video(..., fps_prueba=N)` usa el mismo camino: normaliza el tramo a N fps (`normalizar`), escribe las partes a N fps y toma el audio de la copia. La interfaz (`probar_video`) y la CLI (`--prueba`) usan el fragmento elegido o los primeros `SEGUNDOS_PRUEBA` (10) segundos, `FPS_PRUEBA` (10) por defecto, y guardan `<salida>_prueba.mp4`, siempre empezando de cero. Con pocos fps cada imagen puede quedar hasta medio frame corrida respecto del sonido (50 ms a 10 fps).

7. Carpetas, modelos, configuraciones y ZIP

7.1 Carpeta como destino (Proceso_Carpeta)

`imagenes_carpeta` lista las imágenes del primer nivel en orden de nombre; `procesar_carpeta` las procesa en tandas de `2·hilos` (pocas en memoria), guarda cada una con su nombre en `<carpeta>_output` y anota las que fallan sin cortar el lote.

7.2 Modelos de cara (.npy)

- Vector de 512 valores float32 normalizado: el promedio de los embeddings de las caras usadas (reconocimiento `buffalo_l/w600k_r50`). Se carga sin pickle.

- Al crear, se detecta a 640 px (`DET_SIZE_CREACION`), se descartan caras de calidad 0 y las que no se parecen (< 0.35) a la cara de referencia (la que más se parece a todas).
- Desde video: frames a `--fps` (1 por defecto), como máximo 200 por video, en memoria.
- Archivos: `<nombre>.npy`, `<nombre>.jpg` (miniatura) y `<nombre>.json` (caras usadas y descartadas, fotos, videos, similitud promedio).
- Combinar: promedio ponderado por caras usadas, verificando que cada modelo se parezca al resto (≥ 0.35).

Las caras en primer plano se detectan también al crear modelos (ver 4.7).

7.3 Configuraciones (JSON)

Archivo `configuraciones/<nombre>.json` con `nombre`, `version`, `guardado` y `ajustes`. Cada valor se valida al guardar y al cargar; las opciones que faltan o son inválidas se dejan como estaban.

```
{
  "nombre": "catrina", "version": 1, "guardado": "2026-10-25T18:30:00",
  "ajustes": {
    "asignacion": "posicion", "genero": "todos", "personas_fps": 1.0, "personas_max": 300,
    "indices_fuente": "0", "indices_destino": "0", "mascara": "amplia", "color": "suave",
    "ojos": true, "boca": false, "nariz": false, "frente": false, "maquillaje": 0.6,
    "oclusion": true, "oclusion_modo": "adaptable", "filtro": "adaptativo",
    "mejorar": true, "fuerza_mejora": 0.8, "minimo_mejora": "96", "hilos": 2,
    "contenedor": "mkv", "codec": "libx264", "zip": false, "zip_conservar": false,
    "carpeta_salida": "output", "nombre_archivo": "resultado"
  }
}
```

7.4 ZIP con contraseña (core/Empaquetar.py)

pyzipper con cifrado WinZip AES-256 y sin recomprimir (los videos e imágenes ya vienen comprimidos). Se escribe a un temporal, se vuelve a abrir con la contraseña, se verifica el CRC (`testzip`) y el tamaño de cada archivo, y recién entonces se reemplaza el destino y se borran los sueltos (salvo `conservar`).

8. Interfaz y CLI

8.1 app.py

- `FaceSwapApp`: ventana principal; las opciones se leen en el hilo de la interfaz (`opciones_cambio`) y se fijan al iniciar el proceso (`opciones_proceso`), así cambiarlas a mitad de un video no lo altera.
- `PreviewWindow`: procesa en segundo plano un frame a la vez; si llega otro pedido, solo se guarda el último. Vistas Deslizar, Lado a lado y Solo resultado; máscara de la zona cambiada.
- `VentanaFragmento`: inicio y fin con deslizadores y un reproductor.
- `DropFrame`: zonas de arrastrar y soltar (tkinterdnd2).

8.2 cli.py

- `build_parser` define los comandos; `main` aplica `--config` (como valores por defecto, que el comando puede reemplazar) y `--save-config`.
- `_firma_opciones` arma la firma del video a partir de los argumentos, sin las opciones que no cambian la imagen (salida, hilos, zip, config…) y con marcas para mantener compatibles los videos empezados con versiones anteriores.
- `_ModoIdentidad` encapsula el registro de identidades y el análisis de personas.

9. Rendimiento

Medida	Valor	Dónde
Arranque de la interfaz	~6-7 s (≈ 46 s la primera vez tras reiniciar: lectura de ~2 GB de modelos)	Laptop, CPU
Conservar partes (ojos, 1080p)	236 → 4 ms por cara	Laptop, CPU
Análisis de personas	~8-15 s por minuto de video; máx. ~1,5 min	Laptop, CPU (estimado)
Velocidad en T4 (estimado)	8-15 frames/s con 1 cara; 5-9 con 2 (oclusión + GPEN)	Colab, sin medir
Aceleración con GPU NVIDIA	5-15 veces respecto de CPU (estimado)	—

10. Pruebas realizadas

Las pruebas usan datos sintéticos (frames de ruido, videos de ffmpeg, caras y embeddings simulados) y caras generadas por IA; nunca fotos de usuarios.

Área	Qué se verificó
Conservar partes optimizado	4.200 casos (5 resoluciones, caras en bordes, inclinadas, chicas, grandes, con y sin 106 puntos, todas las combinaciones): imagen y máscara idénticas
Modo de oclusión	debil idéntico a la versión anterior; adaptable y fuerte con 0 % de cara dentro del objeto
Nitidez sin oclusión	Fuera de la zona pegada, 0 píxeles cambiados; caminos correctos con y sin oclusión
Una fuente en varias caras	6 casos de pares, incluidos índices inexistentes y frames con menos caras
Análisis de personas	Numeración por apariciones; extras al final; persona muy girada reconocida con varias tomas y no con una; caché; unión de grupos; cancelar
Modelos desde video	Cantidad de frames por fps, tope repartido en todo el video, descarte de otra persona
Carpetas	Orden, nombres, errores, hilos, detener, CLI con --overwrite
Configuraciones	Ida y vuelta interfaz ↔ JSON, valores inválidos, archivos viejos y dañados
ZIP	AES, contraseña correcta e incorrecta, contenido idéntico, conservar, imagen y carpeta
Sincronía de audio	Videos con destello y pitido en cada segundo: ritmo fijo, ritmo variable y audio corrido; completos, fragmentos y videos de prueba
Caras inclinadas	Cara girada grado a grado de −180° a +180°: error y saltos de los puntos frente a su posición exacta
Video de prueba	fps, cantidad de frames, duración, nombre _prueba, salida normal intacta; botón de punta a punta
Colab	Celdas 4-7 ejecutadas en local con Colab simulado: imagen, video con fragmento, carpeta con ZIP, continuar con otras opciones, caída de h264_nvenc a libx264

11. Dependencias y entorno

Paquete	Uso
numpy, opencv-python, pillow	Imágenes y video
insightface	Analizador buffalo_l e inswapper
onnxruntime-gpu (o onnxruntime)	Ejecución de los modelos
tkinterdnd2	Arrastrar y soltar en la interfaz
pyzipper	ZIP con AES-256
FFmpeg (programa)	Escritura de video, unión y audio

`onnxruntime-gpu` de PyPI está compilado para **CUDA 12** y cuDNN 9. En entornos con CUDA 13 hay que instalar las librerías `nvidia-*-cu12` y una versión de cuDNN compatible con la de ONNX Runtime, o usar un entorno con CUDA 12.

12. Limitaciones conocidas y mejoras pendientes

Tema	Detalle	Propuesta
Índices fuente en la CLI	No se validan antes de empezar (5.1)	Validar como en la interfaz
Resolución del cambio	inswapper trabaja a 128 px	GPEN, o un modelo de mayor resolución
Licencia de inswapper	Uso no comercial	Revisar antes de uso comercial
Colab	Política contra deepfakes; CUDA 13	PC o GPU alquilada; entorno con CUDA 12

13. Cómo extender

- **Una opción nueva del pipeline**: agrégala en `Swapper_Faces.cambiar_una` (en el orden de la figura 2), su parámetro en `cambiar_Faces` y `cambiar_Faces_por_identidad`, su control en `app.py` (y en `opciones_cambio`, `firma_video`, `leer_configuracion`/`aplicar_configuracion`), su argumento en `cli.py` (y en `_CONFIG_A_OPCION`) y su clave en `core/Configuraciones.py`.
- **Compatibilidad de videos a medias**: si la opción cambia la imagen, inclúyela en la firma solo cuando no tenga su valor por defecto anterior, para no invalidar videos empezados antes.
- **Pruebas**: preferir comparaciones exactas contra la versión anterior (como la de 4.200 casos) cuando un cambio no debe alterar el resultado.